

```

module syncFIFO_v2
#(parameter WIDTH = 4,
parameter DEPTH_LEN = 4) // 2^4 depth
(
i_clk, i_rst_n, i_data, wr_en,
rd_en, o_data, o_full, o_empty
);
input i_clk, i_rst_n;
input [WIDTH-1:0] i_data;
input wr_en, rd_en;
output reg[WIDTH-1:0] o_data;
reg [WIDTH-1:0] mem [(1<<DEPTH_LEN)-1:0];
output o_full;
output o_empty;
// points to the address to read/write to
// notice extra bit
reg [DEPTH_LEN: 0] rd_ptr, wr_ptr;
wire rd_req, wr_req;
assign rd_req = rd_en && !o_empty;
assign wr_req = wr_en && !o_full;
wire [DEPTH_LEN: 0] fill;
assign fill = (wr_ptr - rd_ptr);
assign o_empty = (fill==0);
assign o_full = (fill == {1'b1, {DEPTH_LEN{1'b0}}});
always@(posedge i_clk, negedge i_rst_n)
begin
if(!i_rst_n)
begin
wr_ptr <= 5'h00;
end
else
begin
if(wr_req)
begin
mem[wr_ptr[DEPTH_LEN-1: 0]] <= i_data;
wr_ptr <= wr_ptr + 1'b1;
end
end
end
always@(posedge i_clk, negedge i_rst_n)
begin
if(!i_rst_n)
begin
rd_ptr <= 5'h00;
end
else
begin
if (rd_req)
begin
rd_ptr <= rd_ptr + 1'b1;
end
end
end
// Combinational read
assign o_data = mem[rd_ptr[DEPTH_LEN-1:0]];
/*
* ASSERTIONS Synchronous FIFO
*/
// Reset startup check
property async_rst_startup;
@(posedge i_clk) !i_rst_n |-> ##1 (wr_ptr==0 && rd_ptr == 0 && o_empty);
endproperty
assert property (async_rst_startup)
else $display("rst assertion failed at startup", $time);
// Reset check in general

```

```

property async_rst_chk;
@(negedge i_rst_n) 1'b1 |-> ##1 @(posedge i_clk) (wr_ptr==0 && rd_ptr == 0 && o_empty);
endproperty
assert property (async_rst_chk)
else $display("rst assertion failed: ", $time);
// 1) Check if data written to a location is the same data read when read ptr reaches
the location
sequence rd_detect(ptr);
##[0:$] (rd_en && !o_empty && (rd_ptr == ptr));
endsequence
// 2) Don't write to fifo if full
property dont_write_if_full;
@(posedge i_clk) disable iff (!i_rst_n) wr_en && o_full |-> ##1 wr_ptr == $past(wr_ptr);
endproperty
assert property (dont_write_if_full)
else $display("failed at time p2: ", $time);
// 3) Don't read when empty
property dont_read_if_empty;
@(posedge i_clk) disable iff (!i_rst_n) rd_en && o_empty |-> ##1 $stable(rd_ptr);
endproperty
assert property (dont_read_if_empty)
else $display("failed at time p3: ", $time);
// 4) Write pointer should only increment by 1 on write request
property inc_wr_one;
@(posedge i_clk) disable iff (!i_rst_n) wr_en && !o_full |-> ##1 (wr_ptr-1'b1 == $past(wr_ptr));
endproperty
assert property (inc_wr_one)
else $display("time p4: ", $time);
// 5) Read pointer should only increment by 1 on read request
property inc_rd_ptr;
@(posedge i_clk) disable iff (!i_rst_n) rd_en && !o_empty |-> ##1 (rd_ptr - 1'b1 == $past(rd_ptr));
endproperty
assert property (inc_rd_ptr)
else $display("time p5: ", $time);
Endmodule

```

#For Clearing all previous settings

clear -all

#For coverage purpose

check_cov-init type all-model {branch toggle statement} -toggle_ports_only

analyze -sv12 sync_fifo.sva

elaborate

clock i_clk

reset i_rst_n

check_cov -measure type (coi stimuli proof bound) -time_limit 60s -bg

